



REDLINE DAYLIGHT • PROJECT DOCUMENTATION

MariaDB Provisioning with Ansible

Complete task documentation: automating a full **MariaDB** setup on Linux — packages, a custom configuration deployed under `/opt/mariadb`, an application database, a dedicated application user with scoped privileges, and schema seeding — all through one Ansible Role. **Repeatable, version-controlled, zero manual steps.**

By **Nabawy**

ENGINEERED & DOCUMENTED

PROJECT DOC

Task Walkthrough

Ansible

Automation Engine

MariaDB

10.11.x

Jinja2

Templates

Table of Contents

	The Map — Executive Summary	3		Running It — The Commands	10
01	Objective & End State	4	06	Verifying the Result	11
02	The Tools — What & Why	5	07	Adding New Data	12
03	Project Structure — The Tree	6	08	Viewing the Data — 4 Ways	13
04	Steps 1–3 — Install · Service · Folder	7	09	Gotchas — Lessons From the Trench	14
04·b	Steps 4–5 — Config Deploy + Restart	8		Cheat Sheet	15
04·c	Steps 6–8 — DB · User · Seed	9			

► **TASK [Prerequisites]** before reading

[**DEBUG**] **What this document is:** the full engineering record of one automation task — why each piece exists, what every step does, how to run it, how to verify it, and the traps actually hit along the way. Assumes basic Ansible familiarity (playbooks, roles, handlers, templates). If a concept is fuzzy, this doc still shows the commands and results end to end.



The Map — The Whole Task

One Ansible Role, eight steps, one running MariaDB server. Here's the route — every line links to its section.

The mission is defined in [Section 1 — Objective & End State](#): automate a complete MariaDB setup so it's repeatable and version-controlled — ending with a custom config under `/opt/mariadb`, an application database, a dedicated user, and seeded schema. The cast of tools is listed in [Section 2](#): Ansible as the engine, the **community.mysql** collection for DB/user management, PyMySQL on the target, and Jinja2 templates for rendering config.

The battlefield layout is in [Section 3 — Project Structure](#): a playbook entry point, an inventory, requirements, group_vars for the secret, and the **mariadb** role holding everything. Then the build itself, in three pages: [Steps 1–3](#) (install packages, enable & start the service, create the config folder), [Steps 4–5](#) (deploy my.cnf + initdb.sql from templates, symlink into the OS config path, restart via handler with a flush), and [Steps 6–8](#) (create the database, create the user with scoped grants, import the seed SQL).

Operations: [Section 5 — Running It](#) (collection install + the playbook command with the password extra-var), [Section 6 — Verifying](#) (five checks from service status to seeded rows), [Section 7 — Adding Data](#) (one-off vs permanent), and [Section 8 — Viewing the Data](#) (four ways, including what never to edit). Finally, the scars: [Section 9 — Gotchas](#), and the condensed [Cheat Sheet](#). Route clear. ►

KEY IDEA **The promise of this task:** the same playbook, run on any fresh Linux server, produces the exact same working MariaDB — config, database, user, and data. No memory, no manual steps, no drift.



Objective & End State

The mission: automate a complete MariaDB setup on a Linux server using Ansible — so the whole process is **repeatable**, **version-controlled**, and **doesn't rely on manual steps**.

[**KEY IDEA**] This isn't "install a database." It's "**encode a database setup into code**" — so any server, any time, run once, ends up identical. The value isn't the first run; it's the hundredth run being free.

► **TASK [The End State]** what exists when the playbook finishes

A running MariaDB server with all four of these:

1 Custom configuration

Deployed from a known location — `/opt/mariadb` — charset, port, max connections, InnoDB buffer pool. Not hidden in OS default files.

2 An application database

A dedicated database (default: `myapp_db`) created only if it doesn't already exist.

3 A dedicated application user

(default: `myapp_user` @ `localhost`) with **full privileges on that database only** — not the whole server.

4 Initialized schema & seed data

The database starts with its tables and initial rows from a versioned SQL script.

► **TASK [Why this matters]** manual vs automated

✗ Manual setup

SSH in, remember the package names, edit config by hand, restart, create DB in a client, forget whether you granted the user... Every server becomes its own snowflake. Recovery = archaeology.

✓ This task

One playbook, one command. The server's entire DB setup lives in **Git** — reviewable, revertible, and identical on every machine it touches.

🎓 **TAKEAWAY** The four end-state items are the **acceptance criteria** of the whole task. If verification (Section 6) can prove all four, the automation is done — regardless of how many steps it took.

▶ What tools make this possible? → **Section 2: The Tools**



The Tools — What & Why

Six tools carry this task. Knowing **why each one exists** matters more than knowing their names.

Tool	Purpose
Ansible	The automation engine — runs all the steps idempotently (safe to re-run, changes only what needs changing).
community.mysql collection	Provides the <code>mysql_db</code> and <code>mysql_user</code> modules for database and user management.
community.general collection	Supporting modules used alongside the core set.
MariaDB	The database server itself — version 10.11.x, installed via the OS package manager.
PyMySQL	The Python library that Ansible's mysql modules need on the target host to connect and run SQL.
Jinja2 templates	Used to render <code>my.cnf</code> and <code>initdb.sql</code> from variables — one template, many environments.
mysql client / mysqldump	Manual verification, querying, and backups — the human-side tools after automation finishes.

► TASK [The two most-misunderstood pieces] clarifications

Why PyMySQL on the target?

Ansible is agentless — but its `mysql_db` / `mysql_user` modules execute **on the managed node** and speak to MariaDB through a Python driver. No PyMySQL → the DB tasks fail with a "python mysql module not found" error, even though MariaDB itself is running fine.

Why a "collection"?

`community.mysql` isn't shipped with Ansible core — it's a Galaxy collection that must be installed first (Section 5). Forgetting this = "couldn't resolve module" on the very first DB task.

[**DEBUG** · *The division of labor*] Ansible = **the conductor** (order and idempotency) · MariaDB = **the musician** (the actual server) · PyMySQL = **the translator** between them · Jinja2 = **the copywriter** (renders config per environment) · mysql client/mysqldump = **the inspector** after the show.

TAKEAWAY Three install-time dependencies people forget: the **collections** (on the control node, via `requirements.yml`), **PyMySQL** (on the target, installed by the role itself in Step 1), and MariaDB's own packages. All three are handled by the automation — that's the point.

► Where does all of this live on disk? → [Section 3: Project Structure](#)



Project Structure — The Tree

Every file in the project, and what it's for. This tree **is** the architecture:

```

mariadb_project/
├─ playbook.yml           # Entry point – runs the mariadb role
├─ inventory.ini          # Target host(s)
├─ requirements.yml       # Ansible Galaxy collections needed
├─ group_vars/
│   └─ db_servers/
│       └─ vault.yml.example # Template for storing the DB password securely
├─ roles/
│   └─ mariadb/
│       ├── defaults/main.yml # All tunable variables (db name, user, password...)
│       ├── vars/
│       │   ├── Debian.yml    # apt-based package/service names
│       │   └─ RedHat.yml     # dnf/yum-based package/service names
│       ├── tasks/main.yml    # The 8 provisioning steps
│       ├── handlers/main.yml # Restart handler
│       └─ templates/
│           ├── my.cnf.j2      # Config file template
│           └─ initdb.sql.j2   # DB init/seed script template

```

► **TASK [Reading the tree]** four things to notice

vars/ split by family

`Debian.yml` vs `RedHat.yml` — package and service names differ per distro. The role loads the right file per OS, so the same task code works everywhere.

vault.yml.example

The **.example** suffix is deliberate: it's a template to copy into your own (Ansible-Vault-encrypted) file. The real password never enters Git.

Two Jinja2 templates

`my.cnf.j2` (server config) and `initdb.sql.j2` (schema + seed) — both rendered from variables, both living inside the role's templates/ folder.

defaults/main.yml

Every tunable in one place: db name, user name, password reference, ports, buffer sizes. Override from the playbook or extra-vars without touching the role.

[KEY IDEA] The layout is standard Ansible role convention — which means **any Ansible engineer can navigate this project without a guide**. Convention is documentation that never goes stale.

TAKEAWAY Entry point = `playbook.yml` → calls role `mariadb` → which carries tasks (8 steps), handlers (restart), templates (2 files), OS-specific vars, and defaults. Everything else in the tree is plumbing for that flow.

▶ The tree is static — now the 8 steps that make it run: → **Steps 1–3**



Steps 1–3 — Install • Service • Folder

The foundation phase: get MariaDB on the machine, running, and a home ready for its custom config.

1

Install MariaDB + PyMySQL

The role installs `mariadb-server` and `mariadb-client` (Debian) or `mariadb-server` / `mariadb` (RedHat) via the **OS package manager** — plus **PyMySQL**, so Ansible's `mysql` modules can connect later in Steps 6–8. The OS-specific package names come from the vars/ `Debian.yml` / `RedHat.yml` split.

2

Enable and start the service

The `mariadb` systemd service is **enabled** (starts on boot) and **started** immediately. Both in one task — a fresh server goes from "no database" to "database accepting local connections" in a single step.

3

Create the configuration folder

A directory is created at `/opt/mariadb` to hold the custom config and init script — kept **separate from the OS's default config locations**. Everything this deployment owns lives in one known place.

[After steps 1-3 – the server state]



`mariadb-server` installed

+



PyMySQL installed

+



service: enabled + started

+



`/opt/mariadb/` ready

[**KEY IDEA**] Step 1's quiet hero is **PyMySQL** — installing it now, alongside the packages, means Steps 6–8 won't fail later with "mysql module not found." Front-loading dependencies is what makes the playbook run top-to-bottom without surprises.

[**WATCH OUT**] Why `/opt/mariadb` instead of editing the OS config directly? **Ownership and clarity**. The OS owns `/etc/mysql/...`; this deployment owns `/opt/mariadb`. One folder to back up, inspect, or blow away — no guessing which lines in the default config were "ours."

🎓 **TAKEAWAY** Steps 1–3 are pure preparation: **packages + service + a home directory**. Nothing custom has landed yet — that's Steps 4–5. Keeping setup and configuration as separate steps is what makes failures easy to locate.

▶▶ The custom config lands — and MariaDB actually reads it: → **Steps 4–5**

Steps 4–5 — Config Deploy + Restart

The most interesting pair of steps: render the custom files from templates, make MariaDB actually **read** them, and restart at exactly the right moment.

4 Deploy the config file and init script

- `my.cnf` is rendered from `templates/my.cnf.j2` into `/opt/mariadb/my.cnf` — charset, port, max connections, InnoDB buffer pool size, etc.
- `initdb.sql` is rendered from `templates/initdb.sql.j2` into `/opt/mariadb/initdb.sql`.
- Because MariaDB **doesn't read /opt by default**, the role **symlinks** the config into the distro's real include directory — so the settings actually take effect.

[The symlink bridge – where the config goes per distro]



Same source file, different landing spot per OS family — the vars/ split from Section 3 decides which.

5 Restart MariaDB — via handler, flushed immediately

The config deployment **notifies a handler** that restarts the `mariadb` service — and the playbook **flushes handlers immediately** so the new configuration is **live before the next steps run**. The DB creation in Steps 6–8 must talk to a server that's already loaded the new settings.

[Why the flush matters – the timing]



[**KEY IDEA**] Two classic Ansible patterns working together: the **handler** guarantees a restart happens only when the config actually changed (idempotent runs = no pointless restarts), and the **flush** guarantees the restart lands **before** dependent steps — not at the end of the play, when it's too late.

[**WATCH OUT**] Drop the flush, and Steps 6–8 could run against MariaDB still using the **old** config — for example the wrong charset or buffer settings, silently. The handler fires eventually, but your DB was created under the wrong settings.

🔑 **TAKEAWAY** Step 4 in one line: **render two templates into /opt/mariadb, then symlink my.cnf into the OS include path** (Debian: `mariadb.conf.d/99-custom.cnf` · RedHat: `/etc/my.cnf.d/`). Step 5: **notify restart + flush_handlers** = config live exactly when needed.

▶▶ Server ready and configured — now fill it: database, user, data → **Steps 6–8**

Steps 6–8 — DB • User • Seed

With MariaDB installed, configured, and freshly restarted — the actual database assets get created. All three steps use the `community.mysql` collection modules, all three are idempotent.

6 Create the database

`community.mysql.mysql_db` creates the application database — default name `myapp_db` — **if it doesn't already exist**. Second run? The database is there → status `ok` untouched.

7 Create the user and grant privileges

`community.mysql.mysql_user` creates the application user — default `myapp_user` @ `localhost` — and grants **ALL PRIVILEGES on the new database only**.

[KEY IDEA • Scope matters] "On the new database only — not on the whole server." The app can fully own its own database but cannot touch `mysql.*`, other databases, or server settings. Least privilege, enforced at grant time.

8 Initialize the database

`community.mysql.mysql_db` with `state: import` runs `/opt/mariadb/initdb.sql` against the new database — creating the starting schema/tables and seeding the initial rows. The SQL itself came from the `initdb.sql.j2` template rendered back in Step 4.

[Steps 6-8 - the chain]

6 `mysql_db` create `myapp_db` → 7 `mysql_user` `myapp_user` + grants → 8 `mysql_db` `state: import` `initdb.sql`

Order is mandatory: no database → nowhere to create the user's grants · no user → the import could still work but the app couldn't connect · no import → empty schema.

[PROOF • Idempotency across all three] Run the playbook again tomorrow: the DB exists (`ok`), the user exists with identical grants (`ok`), and the import behavior keeps the state consistent instead of duplicating rows blindly. The whole playbook is safe to re-run — that's the entire design goal.

[WATCH OUT] The `-e mariadb_db_password='...'` you pass at run time (Section 5) is what Steps 7's user creation uses. Lose it or change it between runs → the task reports a change or fails to connect. The password is part of the **state**, not just a one-time secret.

🎓 **TAKEAWAY** The final triad: 6 `mysql_db` create (name: `myapp_db`) · 7 `mysql_user` create + **ALL PRIVILEGES scoped to that DB only** · 8 `mysql_db` `state: import` from `/opt/mariadb/initdb.sql`. Database → user → data, in that order, every run.

🔜 All 8 steps done — time to press the button for real: → [Section 5: Running It](#)

▶ Running It — The Commands

Two commands from a fresh clone to a running database. First the terminal, then what every piece means:

▶ **TASK [The exact commands]** from the project root

```
cd mariadb_project

# Install required collections
$ ansible-galaxy collection install -r requirements.yml

# Run the playbook
$ ansible-playbook -i inventory.ini playbook.yml \
  --ask-become-pass \
  -e mariadb_db_password='YourStrongPassword'
```

▶ **TASK [The inventory note]** pointing at your target

`inventory.ini` should point at the target host — two common shapes:

Local testing

```
localhost ansible_connection=local
```

Runs against the same machine — no SSH involved.

Remote server

```
myserver ansible_host=10.0.0.5
```

A real hostname/IP — uses standard SSH.

▶ **TASK [Every flag decoded]** nothing is magic

-i inventory.ini

which hosts to run against

--ask-become-pass

prompts for the sudo password (package installs need root)

-e var=value

extra-var — injects the DB password into the run

[**DEBUG** · The two commands, in order] **Command 1** stocks your control node with the **community.mysql** and **community.general** collections (Section 2). **Command 2** does the actual work on the target. Skip command 1 → command 2 dies on its first mysql task with "module not found."

[**WATCH OUT**] The `-e` password ends up in your shell history. For anything beyond a lab, prefer the `group_vars/db_servers/vault.yml` route (encrypted with Ansible-Vault — see the project structure, Section 3) and drop the `-e` flag entirely.

🏠 **TAKEAWAY** The run recipe: **cd** → **galaxy collections** → **playbook with inventory + become password + DB password extra-var**. Local test = `ansible_connection=local`; remote = real host in `inventory.ini`.

▶ It ran — but "it ran" isn't "it works." Prove it: → [Section 6: Verifying](#)



Verifying the Result

A finished playbook proves **nothing** — a green recap means the tasks ran, not that the outcome is correct. Five checks, each mapping to one end-state promise from Section 1:

► **TASK [Check 1 — the service is alive]** end state: running server

```
$ sudo systemctl status mariadb --no-pager
• mariadb.service — MariaDB 10.11.x database server
  Active: active (running)
```

► **TASK [Check 2 — the custom config actually applied]** end state: /opt/mariadb config live

```
$ sudo mysql -e "SHOW VARIABLES LIKE 'character_set_server';"
+-----+
character_set_server  utf8mb4    ← proves the symlinked config loaded
```

► **TASK [Checks 3 & 4 — database and user exist]** end states: DB + scoped user

```
$ sudo mysql -e "SHOW DATABASES;"
myapp_db          ← the application database is there

$ sudo mysql -e "SHOW GRANTS FOR 'myapp_user'@'localhost';"
GRANT ALL PRIVILEGES ON `myapp_db`.* TO ... ← scoped, not server-wide
```

► **TASK [Check 5 — seeded data is present]** end state: schema + rows

```
$ sudo mysql myapp_db -e "SELECT * FROM app_info;"
...seeded rows appear... ← Step 8's import actually landed
```

[**KEY IDEA**] Each check targets a **specific step** from Section 4: status → Step 2 · charset variable → Step 4 (config live) · SHOW DATABASES → Step 6 · SHOW GRANTS → Step 7 · SELECT → Step 8. If all five pass, every layer of the automation is proven — not just "it didn't crash."

[**DEBUG · The cleverest check**] Check 2 is the one people skip — and the one that catches the most. MariaDB **starting fine** doesn't mean it read your custom config. Querying a variable you set in `my.cnf.j2` is the only direct proof the symlink bridge worked.

🔑 **TAKEAWAY** The verification set: `systemctl status` · `SHOW VARIABLES` · `SHOW DATABASES` · `SHOW GRANTS` · `SELECT` . Five commands = five end-state promises verified. Keep them in your runbook next to the playbook.

▶ Verified and running — now the day-2 question: how does new data get in? → [Section 7](#)

+ Adding New Data — Two Paths

There are two legitimate ways to put new data into this database — and they serve **different purposes**. Knowing which to reach for is the whole lesson.

► **TASK [Path 1 — one-off, via the mysql client]** quick, manual, not reproducible

```
$ mysql -u myapp_user -p'YourStrongPassword' -D myapp_db \
-e "INSERT INTO app_info (key_name, value) VALUES ('app_name', 'MyApp');"

```

Fast for a quick test row or an ad-hoc fix. But: it lives only on this server, in this moment — a rebuilt machine won't have it.

► **TASK [Path 2 — permanent / repeatable, via the role]** the Ansible way

Add your `CREATE TABLE` / `INSERT` statements to `roles/mariadb/templates/initdb.sql.j2`, then **rerun the playbook** — so the schema stays **reproducible on any server it's deployed to**:

 `edit initdb.sql.j2` →  `rerun playbook` →  `schema reproducible everywhere`

[**KEY IDEA** · The decision rule] **Will this data need to exist on the next server too?** Yes → it belongs in `initdb.sql.j2` (version-controlled, part of the state). No → the mysql client is fine. Everything reproducible goes in code; everything throwaway stays in the client.

[**WATCH OUT**] Remember from Step 8: the import runs against the DB at provision time. A brand-new server gets the **full updated initdb.sql**; an existing server only picks up changes the next time its state is reconciled — for row-level data on live servers, the client (or a migration task) is the operational path.

🔑 **TAKEAWAY** One-off row/test = **mysql client INSERT** · permanent schema/seed = **edit initdb.sql.j2 + rerun**. The second path is why this whole task exists: the data setup travels with the code.

▶ Data is in — where does it physically live, and how do you look at it? → **Section 8**

Viewing the Data — 4 Ways

"Where is my data?" has four different honest answers — each useful in a different situation, and one of them is a trap.

1 The seed file — plain text

```
$ cat /opt/mariadb/initdb.sql
```

The **source** that seeded the DB — readable, versioned, rendered from the template. Read this to see *what was supposed to be created*.

2 On-disk storage — the trap ⚠

```
/var/lib/mysql/<db_name>/
```

Binary `.ibd` files — **not human-readable; never edit these directly**. This is where the data physically lives, but it belongs to MariaDB, not to you.

3 Readable full snapshot

```
$ sudo mysqldump myapp_db > myapp_db_backup.sql
```

Schema + **data** exported as readable SQL — the proper way to capture the complete current state (also your backup move).

4 Querying directly

```
$ sudo mysql myapp_db -e "SELECT * FROM <table>;"
```

The everyday move — ask the live server a question, get the live answer.

► **TASK [Choosing a viewer]** which one when?

You want to...	Reach for
See what the automation intended to create	1 <code>cat initdb.sql</code>
Back up or hand off the full current state	3 <code>mysqldump</code>
Check rows right now	4 <code>mysql -e "SELECT..."</code>
Satisfy curiosity about storage internals	2 <code>/var/lib/mysql/</code> — look, never touch

[WATCH OUT • Emphasized] Editing files under `/var/lib/mysql/` while MariaDB is running (or ever, really) is how databases **die**. The server owns those bytes. Your interface to the data is always SQL — through the client, or a dump.

🔑 **TAKEAWAY** Four views: **seed source** (`initdb.sql`, text) · **raw storage** (`/var/lib/mysql`, binary — hands off) · **full snapshot** (`mysqldump`, readable SQL) · **live query** (`mysql -e`). Each answers "where is my data" from a different altitude.

🔗 Before you go — the three mistakes this task actually produced: → [Section 9: Gotchas](#)



Gotchas — Lessons From the Trench

Three things that actually went sideways while building this task. Each one costs minutes (or hours) the first time — reading them here costs you seconds.

[**GOTCHA 1** · **Two interpreters, one prompt**] The `mysql` client and the bash shell are **two different command interpreters**. SQL pasted into bash — or shell commands pasted into an open `mysql` session — produces **confusing, unrelated errors**. Always check the prompt before typing:



\$ means bash

Shell commands live here: `sudo systemctl`, `cat`, `mysql`
`-e "..."`. SQL typed here = `syntax error` nonsense.



MariaDB [db]> means mysql

SQL lives here: `SELECT...`, `SHOW...`. Shell commands typed here = `ERROR 1064` nonsense.

[**GOTCHA 2** · **Multi-line SQL**] Multi-line SQL is safest **saved to a .sql file and piped in** (`mysql ... < file.sql`) rather than pasted interactively. Pasting long statements into the client invites partial-line errors, lost semicolons, and copy bugs — the file is repeatable, the paste is a gamble.

[**GOTCHA 3** · **bind-address = 127.0.0.1**] The deployed config sets `bind-address = 127.0.0.1` — the server **only accepts local connections**. This is **intentional for a dev/local setup**: the database simply isn't reachable from other machines. If remote access is ever needed, change the bind address **and** open the firewall — both, or it still won't answer.

[**Remote access checklist — only when needed**]

1 change bind-address

+

2 open the firewall

+

3 user host ≠ localhost

→

reachable

Note the 3: the default grant is `myapp_user@localhost` — a remote client needs a matching user/host too.

[**KEY IDEA**] All three gotchas share one root: **the error appears far from its cause**. Wrong interpreter, wrong input channel, wrong bind address — each one looks like a totally different problem than it is. Slow down, check the prompt, check the file, check the config.

🏠 **TAKEAWAY** The trench notes: **1.** prompt tells you the language (`$` = bash, MariaDB `[>]` = SQL) · **2.** pipe multi-line SQL from a file · **3.** bind-address `127.0.0.1` = local-only by design; remote needs config + firewall + user host.

▶ All sections complete — the condensed reference: → [Cheat Sheet](#)

The Cheat Sheet — print & pin it

- ▶ **THE TASK:** Ansible provisions MariaDB — 8 steps, one role, fully reproducible
- ▶ **END STATE:** running server + /opt/mariadb config + myapp_db + myapp_user + seeded schema
- ▶ **THE FLOW:** install → service → folder → templates+symlink → restart(flush) → db → user → import

[⚙️ Run & Verify]

```
$ ansible-galaxy collection install -r requirements.yml
$ ansible-playbook -i inventory.ini playbook.yml \
  --ask-become-pass \
  -e mariadb_db_password='...'

# verify (5 checks):
$ sudo systemctl status mariadb
$ sudo mysql -e "SHOW VARIABLES LIKE 'character_set_server';"
$ sudo mysql -e "SHOW DATABASES;"
$ sudo mysql -e "SHOW GRANTS FOR 'myapp_user'@'localhost';"
$ sudo mysql myapp_db -e "SELECT * FROM app_info;"
```

[🗃️ Data Access]

```
$ cat /opt/mariadb/initdb.sql # seed source
$ sudo mysqldump myapp_db > backup.sql # snapshot
$ sudo mysql myapp_db -e "SELECT..;" # live query
/var/lib/mysql/ - binary, NEVER edit
```

[📁 Key Paths]

playbook.yml	entry point
inventory.ini	targets (local: ansible_connection=local)
roles/mariadb/tasks/	the 8 steps
templates/my.cnf.j2	server config source
templates/initdb.sql.j2	schema + seed source
/opt/mariadb/	deployed config + init script
/etc/mysql/mariadb.conf.d/99-custom.cnf	Debian symlink
/etc/my.cnf.d/	RedHat symlink target

[🚀 The 8 Steps]

- 1 install mariadb + PyMySQL
- 2 enable + start service
- 3 create /opt/mariadb
- 4 deploy my.cnf + initdb.sql + symlink
- 5 restart via handler + flush
- 6 mysql_db → create myapp_db
- 7 mysql_user → myapp_user + grants
- 8 mysql_db import → initdb.sql

bind-address 127.0.0.1 = local-only

\$ = bash • MariaDB [>] = SQL

pipe SQL from .sql files

[**KEY IDEA** • Closing line] One role, eight steps, five verification checks — and the same MariaDB server comes out the other side every single time. That's infrastructure as code doing exactly what it says. ▶